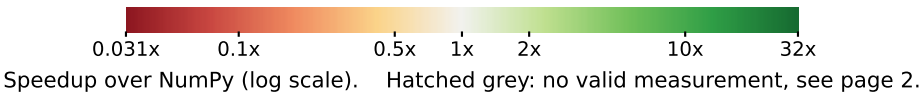


Pluto vs DaCe on PolyBench-derived NPBench kernels

Median speedup over NumPy (50 repetitions)

	Pluto	DaCe auto_opt	NumPy	
geo-mean	16.0x	7.3x	758.7 ms	
gemm	0.24x	2.0x	26.7 ms	linear-algebra / bias
gemver	29.4x	19.0x	183.3 ms	
syrk	166x	311x	2.87 s	
syr2k	248x	312x	6.82 s	
trmm	531x	99.9x	1.74 s	
atax	0.87x	0.88x	20.3 ms	linear-algebra / kernels
bicg	0.84x	0.91x	20.0 ms	
mvt	0.68x	0.81x	13.4 ms	
cholesky	42.7x	7.9x	2.57 s	linear-algebra / solvers
durbin	2.4x	0.69x	295.6 ms	
gramschmidt	12.9x	0.24x	47.3 ms	
lu	79.9x	3.7x	7.81 s	
ludcmp	3.6x	3.8x	8.05 s	
trisolv	15.3x	1.0x	103.6 ms	datamining
correlation	3.1x	0.74x	29.4 ms	
deriche	2.5x	2.3x	1.12 s	medley
floyd_warshall	6.5x	2.5x	14.01 s	
nussinov	658x	602x	7.13 s	stencils
adi	declined	0.20x	360.4 ms	
fdtd_2d	55.0x	76.3x	2.25 s	
heat_3d	19.1x	168x	10.24 s	
jacobi_2d	188x	304x	50.81 s	
seidel_2d	103x	7.2x	5.88 s	



Pluto vs DaCe on PolyBench-derived NPBench kernels: detailed results

Preset paper, 50 repetitions per kernel. Every blank cell on page 1 is accounted for below; a speedup is given only where the result validated.

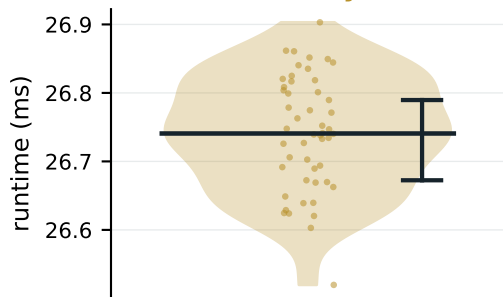
DaCe column: dace_cpu_autoopt / extended@eb7b1352a
Pluto: polycc --tile --parallel --codegen-context=1 (Clan frontend), clang -O3 -march=native

kernel	NumPy runtime	Pluto	DaCe auto_opt	validated note / reason a cell is blank	
gemm	26.7 ms	0.24x	2.0x	yes	both columns validated against the NumPy reference
gemver	183.3 ms	29.4x	19.0x	yes	both columns validated against the NumPy reference
syrk	2.87 s	166x	311x	yes	both columns validated against the NumPy reference
syr2k	6.82 s	248x	312x	yes	both columns validated against the NumPy reference
trmm	1.74 s	531x	99.9x	yes	both columns validated against the NumPy reference
atax	20.3 ms	0.87x	0.88x	yes	both columns validated against the NumPy reference
bicg	20.0 ms	0.84x	0.91x	yes	both columns validated against the NumPy reference
mvt	13.4 ms	0.68x	0.81x	yes	both columns validated against the NumPy reference
cholesky	2.57 s	42.7x	7.9x	yes	both columns validated against the NumPy reference
durbin	295.6 ms	2.4x	0.69x	yes	Pluto: correct and tiled, but polycc marked no loop parallel -- this is a SINGLE-THREADED result. The kernel is an inherently sequential recurrence and both polycc frontends agree there is no parallelism to find, so the speedup is not comparable to the 72-thread rows above.
gramschmidt	47.3 ms	12.9x	0.24x	yes	both columns validated against the NumPy reference
lu	7.81 s	79.9x	3.7x	yes	both columns validated against the NumPy reference
ludcmp	8.05 s	3.6x	3.8x	yes	Pluto: correct and tiled, but polycc marked no loop parallel -- this is a SINGLE-THREADED result. The kernel is an inherently sequential recurrence and both polycc frontends agree there is no parallelism to find, so the speedup is not comparable to the 72-thread rows above.
trisolv	103.6 ms	15.3x	1.0x	yes	both columns validated against the NumPy reference
correlation	29.4 ms	3.1x	0.74x	yes	both columns validated against the NumPy reference
deriche	1.12 s	2.5x	2.3x	yes	both columns validated against the NumPy reference
floyd_warshall	14.01 s	6.5x	2.5x	yes	both columns validated against the NumPy reference
nussinov	7.13 s	658x	602x	yes	Pluto: SINGLE-THREADED. polycc's transformation is correct and tiled, but the `omp parallel for` it puts on the i loop is not: that loop reads rows below i which later iterations write. The transform is used exactly as generated and compiled without -fopenmp, so the pragma is inert. Validated 27/27 over three presets and 1/8/72 threads.
adi	360.4 ms	declined	0.20x	DaCe only	Pluto: Two defects. Clan transposes adi's scop parameters: its domain for v[0][i] says -t+N>=0 and -i+TSTEPS-2>=0 where the loops are t=1..TSTEPS and i=1..N-1, so polycc guards on TSTEPS and can write past the arrays. Hoisting the scalar setup fixes that, but polycc then still emits the back-substitution before the loop producing its p and q.
fdtd_2d	2.25 s	55.0x	76.3x	yes	both columns validated against the NumPy reference
heat_3d	10.24 s	19.1x	168x	yes	both columns validated against the NumPy reference
jacobi_2d	50.81 s	188x	304x	yes	both columns validated against the NumPy reference
seidel_2d	5.88 s	103x	7.2x	yes	both columns validated against the NumPy reference

gemm (n = 50)

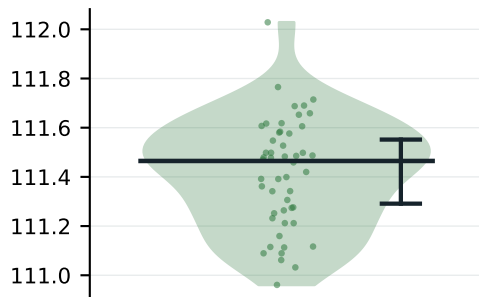
Dense matrix multiply. NumPy and DaCe reach tuned BLAS; Pluto transforms the loop nest.

NumPy



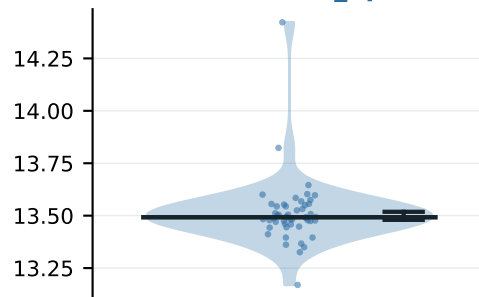
median 26.74 ms
95% CI [26.67, 26.79]
lag-1 acf +0.53

Pluto



median 111.5 ms
95% CI [111.3, 111.6]
lag-1 acf +0.52

DaCe auto_opt

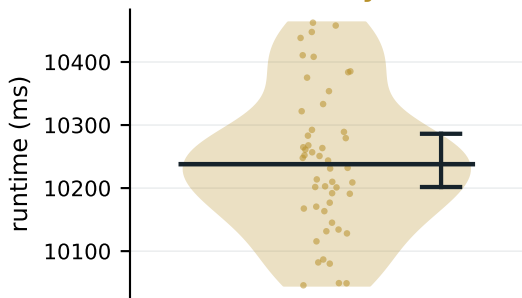


median 13.49 ms
95% CI [13.48, 13.52]

heat_3d (n = 50)

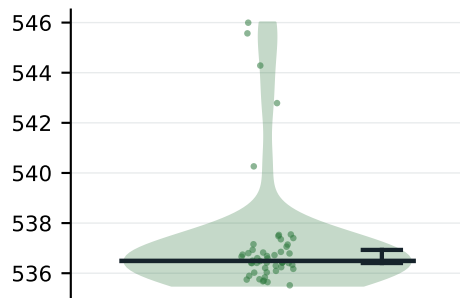
Regular affine stencil: the standard case for polyhedral tiling and parallelisation.

NumPy



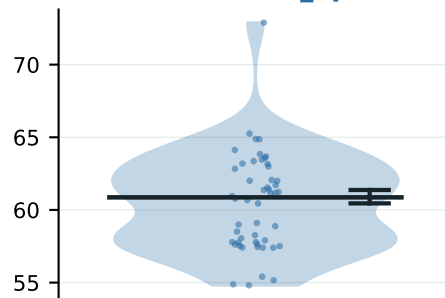
median 1.024e+04 ms
95% CI [1.02e+04, 1.029e+04]
lag-1 acf +0.66

Pluto



median 536.5 ms
95% CI [536.4, 536.9]

DaCe auto_opt



median 60.87 ms
95% CI [60.45, 61.37]
lag-1 acf -0.58